

Cetus DLMM Audit

audited by Asymptotic

November 21, 2025

Summary

Asymptotic conducted a security audit of the Cetus DLMM protocol (packages: `cetus_dlmm` and `dlmm_router`).

Key Findings:

- **5 Low severity issues** — all remediated (input validation, bounds checking, semantic inconsistencies)
- **14 Advisory findings** — 10 remediated, 4 acknowledged (code quality and optimization opportunities)

The codebase demonstrates strong security fundamentals with no exploitable vulnerabilities. All material security issues have been successfully addressed. The remaining acknowledged advisories represent acceptable trade-offs and do not pose security risks to the protocol.

Audited code

We audited the `cetus_dlmm` repository, which contains two packages:

- `cetus_dlmm`
- `dlmm_router`

The initial commit reflects the version at the start of the audit, and the final commit reflects the version integrating all fixes:

- initial commit: `https://github.com/CetusProtocol/cetus-dlmm/commit/d1037ffc7ff7ffd3144`
- final commit: `https://github.com/CetusProtocol/cetus-dlmm/commit/974933c29a1705186e378`

Legend

Issue severity

- **Critical** — Vulnerabilities which allow account takeovers or stealing significant funds, along with being easy to exploit.
- **High** — Vulnerabilities which either can have significant impact but are hard to exploit, or are easy to exploit but have more limited impact.
- **Medium** — Moderate risks with notable but limited impact.
- **Low** — Minor issues with minimal security implications.
- **Advisory** — Informational findings for security/code improvements.

Legal Disclaimer

Terms and Conditions and Liability

This report is subject to the terms and conditions—including liability limitations—established between Asymptotic and the paying entity. By sharing code with Asymptotic, developers acknowledge and agree to these conditions. In the absence of executed agreements, liability is limited to the fees paid for these services.

Scope

This security audit report focuses specifically on reviewing the Move smart contract code. The audit does not analyze or make any claims about other components of the system, including but not limited to:

- Frontend applications and user interfaces
- Backend services and infrastructure
- Off-chain components and integrations
- Deployment procedures and operational security
- Third-party dependencies and external services

Limitations

The findings and recommendations in this report are limited to the Move code implementation and its immediate interactions within the Sui blockchain environment.

Creating proofs for specifications is on a "best effort" basis. We manually audited the code in instances where formal proofs were not technically feasible.

L-1: Missing Validation and Performance Optimization in `add_group_if_absent`

Severity: Low

Description

The function `add_group_if_absent` accepts `group_index` without validation and can create bins with IDs outside the valid range. The public wrapper in the `pool` module exposes this to external callers without additional checks.

Additionally, for each new group, the function creates 16 bins by calling `default_bin` → `get_price_from_id` → `pow(base, bin_id)` sixteen times. This performs 16 expensive exponential calculations.

Recommendation

Calculate the corresponding score from `group_index` and validate that it falls within acceptable bounds.

Consider optimizing performance by replacing power function with multiplication: calculate base price for the first bin and for each subsequent bin, multiply the current price by the base value. This incremental multiplication maintains the mathematical relationship between consecutive bins while avoiding repeated exponential calculations. This reduces gas costs for group creation by approximately 60 to 80 percent.

Remediation

✓ **Commit:** 05a78aa

L-2: No Bounds Validation in `set_min_reward_duration`

Severity: Low

Description

The function `set_min_reward_duration` in the `config` module accepts any `u64` value without validation. When `min_reward_duration` is set to a value near max `u64`, subsequent calls to `add_reward` will abort as there is no such `end_time` to calculate duration $\geq$ `min_reward_duration`.

There is also no upper bound on reward duration in `add_reward`, allowing managers to create reward periods spanning decades or centuries. This effectively locks reward tokens for impractically long periods, reducing capital efficiency and creating zombie rewards that will never realistically complete their emission schedule.

Recommendation

Implement validation bounds for both configuration and usage:

- Add minimum and maximum duration constants
- Use them as bounds for `min_reward_duration`
- Enforce maximum duration in `add_reward`

Remediation

✓ **Commit:** 05a78aa

L-3: Incorrect Semantics for `unblock(ALL)` Operation

Severity: Low

Description

The restriction system treats `ALL` as an independent role bit rather than a composite of all operation types. When `unblock(entity, ALL)` is called after specific operations were blocked, those specific blocks persist. The `unblock` function only removes the `ALL` bit, leaving individual operation bits intact.

However, the `is_blocked` function correctly checks:

- If `ALL` bit is set → return blocked
- OR if specific operation bit is set → return blocked

Recommendation

Implement proper `ALL` operation semantics where `ALL` represents the union of all operations or consider removing entity entirely in case of unblocking all operations.

Remediation

✓ **Commit:** 05a78aa

L-4: Missing Input Validation in `bin_id_from_score`

Severity: Low

Description

The public function `bin_id_from_score` converts a score to a bin ID without validating the input, creating an asymmetry with its inverse function `bin_score` which enforces strict validation.

Recommendation

Add input validation matching the constraints enforced by `bin_score`: ensure that score falls within a valid and expected range before conversion.

Remediation

✓ **Commit:** 05a78aa

L-5: Inconsistent Validation in `pool::add_reward`

Severity: Low

Description

The `pool::add_reward` function is responsible for adding rewards to the pool, but it exhibits several inconsistencies.

Managers (reward manager role) can bypass critical validations that non-managers must follow. This could lead to inconsistencies if manager actions deviate from expected standards.

Start time validations differ between pool and reward manager levels, causing uncertainty about the correct requirements.

Neither `pool::add_reward` nor `reward::add_reward` validates that the reward amount is greater than zero.

Recommendation

Apply consistent validation logic across all roles and levels, ensuring universal constraints with clear documentation. Enforce minimum rewards amount at least greater than zero.

Remediation

✓ **Commit:** 05a78aa

Client confirmed the difference between manager and user checks is intentional. Regular users must align reward start/end times with weekly periods to prevent excessive `period_emission_rates` node. Managers may specify custom times for product-level flexibility. Aside from that, everything is addressed.

A-1: Theoretical Collision Risk in `new_pool_key`

Severity: Advisory

Description

The function `new_pool_key` generates pool identifiers by concatenating two type names without encoding their boundary. This theoretically allows different type pairs to produce identical concatenations.

Hypothetical short type names (not actual Sui format):

- Pool 1: `TypeA = "DEF"`, `TypeB = "ABC"` → Concatenation = `"DEFABC"`
- Pool 2: `TypeA = "DEFAB"`, `TypeB = "C"` → Concatenation = `"DEFABC"`

Both pools would generate the same result, creating a collision.

Sui type names have the format: `<package_address>::<module>::<type>`, which makes such collisions practically impossible. But if type name formatting ever changes, vulnerability could become exploitable.

Also formal verification cannot mathematically prove collision-resistance without examining Move compiler internals.

Recommendation

Add explicit length encoding to make collision mathematically impossible regardless of type name format (for at least one type).

Remediation

Acknowledged — The client responded that directly deploying this change could result in duplicate pools, which is not necessary at present. The issue is only possible if Sui Move itself changes the type name formatting standard. This is very unlikely so it is reasonable to only apply a more comprehensive solution later if ever needed. We consider this secure as is.

A-2: Code Duplication in Liquidity Management

Severity: Advisory

Description

The liquidity management system contains significant code duplication between position opening and liquidity addition operations inside the `pool` module:

- `OpenPositionCert` and `AddLiquidityCert` are ∼90% identical (16 shared fields). Only differences: `width` and `next_bin_id` fields in `OpenPositionCert`.
- `repay_open_position` and `repay_add_liquidity` share ∼90% identical code. Only difference: 3 lines for width validation in open variant.
- `open_position_on_bin` and `add_liquidity_on_bin` differs only by sequential bin validation and range checks in open variant.
- `new_open_position_cert` and `new_add_liquidity_cert`
- `open_position` and `add_liquidity`

Code duplication introduces maintenance risks and should be refactored. However, if the duplicated code is already in production and well tested, the refactoring can be postponed until a major protocol update.

Recommendation

Apply the composition pattern to eliminate duplication. Use the `AddLiquidityCert` struct as part of `OpenPositionCert`, keeping specific fields like `width` and `next_bin_id` in the `OpenPositionCert`.

Then refactor position-related functions to use their liquidity counterparts, ensuring that all validations are performed at the appropriate level.

Remediation

Acknowledged — The client noted that the current implementation is already in production and has been thoroughly tested. To minimize potential risks, the refactoring will be scheduled for a future major protocol update.

A-3: Redundant Loop Condition in `prev_score_in_group`

Severity: Advisory

Description

The while loop in the `prev_score_in_group` function uses `while (offset >= 0)` where `offset` is type `u8`. Since unsigned integers can never be negative, this condition is always true.

Also the function contains a 10-line if-else block handling the `offset_in_group == 0` case, but most of this complexity is unnecessary. The loop can naturally handle the include case, and only the underflow case requires special handling to prevent `offset = 0 - 1` underflow.

Recommendation

Simplify the function by removing redundant logic: remove redundant loop condition and replace if-else block with edge case processing only:

```
if (!include && offset_in_group == 0) {
    return option_u64::none()
};
```

Remediation

✓ **Commit:** 05a78aa

A-4: Missing Helper Function for Score Composition

Severity: Advisory

Description

The codebase contains at least 5 instances where scores are manually calculated from group index and offset using the formula `group_index * 16 + offset`. This duplicates logic across multiple functions without a corresponding helper function, despite having the inverse operation `resolve_bin_position` that decomposes scores into their components.

Recommendation

Introduce a helper function (e.g., `compose_score`) for score calculation. This creates symmetry with `resolve_bin_position`, improves code clarity, and centralizes the score composition logic in a single location.

Remediation

✓ **Commit:** 05a78aa

A-5: Mixed Tracking of Admin and User Withdrawals in `reward_harvested`

Severity: Advisory

Description

The function `emergency_withdraw_refund_reward` withdraws refunded rewards by calling the internal `withdraw` function, which increments `reward.reward_harvested`. This mixes two fundamentally different types of withdrawals: user harvests and admin emergency withdrawals.

Also the documentation explicitly states `reward.reward_harvested` tracks user harvests, but admin withdrawals are included.

Recommendation

Add a separate tracking field for admin emergency withdrawals to maintain clear accounting or document existing value contain both types of withdrawals.

Remediation

✓ **Commit:** 05a78aa

The logic remains unchanged, but the comment was updated to reflect the logic.

A-6: Redundant Field Assignment in `add_liquidity`

Severity: Advisory

Description

The `add_liquidity` function in the `position` module contains redundant field assignments. The function calls `new_bin_stat` which already initializes `BinStat` fields with the bin's global growth values, then immediately overwrites these exact same fields with identical values.

Recommendation

Remove the redundant field assignments.

Remediation

✓ **Commit:** 05a78aa

A-7: Inefficient Data Structure for `reward_white_list`

Severity: Advisory

Description

The `RewardConfig` struct uses `VecMap<TypeName, bool>` for `reward_white_list`, but the boolean values are meaningless. The code always inserts `true` and only checks for key presence using `contains()`, never reading the boolean value itself.

The boolean value carries no semantic meaning — the key's presence itself indicates whitelist membership.

Recommendation

Replace `VecMap<TypeName, bool>` with `VecSet<TypeName>` throughout the codebase. `VecSet` is semantically correct for set membership and more efficient.

Remediation

Acknowledged — The client stated that the contract is already deployed and operating stably. Therefore, no changes will be implemented at this stage.

A-8: Lack of Start Time Validation in `create_partner`

Severity: Advisory

Description

The function `create_partner` does not validate that `start_time` is not in the past, while the test function `create_partner_for_test` does enforce this check. Partners with past start times become immediately active, which can distort analytics and historical tracking.

Recommendation

Enforce the same validation as the test function (reject any `start_time` in the past) or auto-adjust to current time (if the provided `start_time` is in the past, automatically use the current time instead).

Remediation

✓ **Commit:** 05a78aa

A-9: Unnecessary Complexity in `is_right_order`

Severity: Advisory

Description

The function `is_right_order` contains unnecessary complexity. Once it determines that `byte_a > byte_b` at line 393, it sets `check_pass = true` and continues looping through all remaining bytes. However, at this point the ordering is already determined and the function can return immediately.

Recommendation

Simplify the function by returning immediately when the order is determined (remove `check_pass` entirely).

Remediation

✓ **Commit:** 05a78aa

A-10: No Early Return in `reward_index`

Severity: Advisory

Description

The function `reward_index` in the `reward` module does not return early when it finds the matching reward type. Instead, it continues iterating through all remaining elements in the vector, performing unnecessary comparisons.

Recommendation

Add an early return when a match is found, even though the number of rewards is currently limited to 5.

Remediation

✓ **Commit:** 05a78aa

A-11: Unnecessary Mutable Parameter Usage

Severity: Advisory

Description

Three public functions declare parameters as mutable (`&mut`) but only perform read operations on them. This violates Move's principle of least privilege and can mislead developers about a function's side effects:

- `registry::create_pool(..., config: &mut GlobalConfig,)` only calls read-only methods and passes `config` as immutable `&GlobalConfig` to internal function
- `partner::update_time_range(..., ctx: &mut TxContext)` only calls `tx_context::sender(ctx)` which is a read operation
- `config::emergency_unpause(config: &mut GlobalConfig, ...)` only reads `config.before_ve` for validation

Using `mut` unnecessarily can introduce avoidable borrow-checker constraints in the calling code.

Recommendation

Change all three parameters from mutable to immutable references.

Remediation

✓ **Commit:** ececd18

A-12: Gas Waste Due to Missing No-Op Guards

Severity: Advisory

Description

Several functions lack checks to prevent redundant state updates or unnecessary executions when inputs match existing values or when operation amounts are zero. These no-op operations do not impact protocol logic but still consume gas and increase transaction costs:

- `partner.move:update_ref_fee_rate` → no check if `new_fee_rate == ref_fee_rate`
- `partner.move:update_time_range` → no check if `start_time` or `end_time` unchanged
- `reward.move:emergency_pause` and `emergency_unpause` → no check if already in same state
- `reward.move:make_public` and `make_private` → no check if already in target state
- `config.move:set_min_reward_duration` and `set_manager_reserved_reward_init_slots` → no check if value unchanged
- `pool.move:collect_position_fee` → no check if `fee_a == 0 && fee_b == 0`
- `pool.move:collect_position_reward` → no check if `amount == 0`

Recommendation

Add early-return guards to skip execution when parameters or states are unchanged, or when computed values (e.g., reward amounts or fees) are zero.

Optionally, introduce a `min_amount` threshold parameter to ensure execution only when economically meaningful (for functions like `collect_position_fee`).

Remediation

✓ **Commit:** 05a78aa

The client explained that `collect_position_fee` and `collect_position_reward` intentionally allow zero-value calls to maintain compatibility with the frontend and vault integrations. These functions remain callable even when fees or rewards are zero, preventing unnecessary transaction failures.

All other recommendations have been implemented, except for `set_manager_reserved_reward_init_sl` which is rarely used and of negligible effect.

A-13: Documentation / Comment Mismatches

Severity: Advisory

Description

Multiple functions and struct fields have comments that don't match the actual implementation, creating confusion and potential for errors during maintenance or audits:

- `parameters.move:124` comment says `filter_period < t < decay_period` but code uses `>=`
- `dlmm_math.move:81, 86` (`calculate_amount_in`) comments say `[Rounding::Down]` but code uses `mul_div_ceil`
- `dlmm_math.move:145, 170, 296` (fee functions) comments say "fee rate in basis points" but actual unit is where `1_000_000_000 = 100%`
- `reward.move` documentation missing `emergency_reward_pause` field entirely

Recommendation

Update all incorrect comments to match implementation.

Remediation

✓ **Commit:** 05a78aa

A-14: Code Duplication in Liquidity Distribution Strategies

Severity: Advisory

Description

The `dlmm_router` package contains three separate liquidity distribution strategy implementations (`spot.move`, `curve.move`, `bid_ask.move`) with extreme code duplication. Unique differences: only 5–10 lines per strategy.

The only meaningful differences are:

- `spot.move`: Uses uniform weight (1.0) for all bins
- `curve.move`: Uses decreasing weight formula (`MAX_WEIGHT - distance × decay`)
- `bid_ask.move`: Uses increasing weight formula (`MIN_WEIGHT + distance × growth`)

Both `open_position` and `add_liquidity` functions contain extensive logic. It is recommended to modularize their implementation by moving reusable or complex parts into dedicated helper functions to enhance clarity and reduce complexity.

Recommendation

Replace `spot.move`, `curve.move`, and `bid_ask.move` with a unified `strategies.move` implementation:

- Add `Strategy` enum to represent the three distribution types (`Spot`, `Curve`, `BidAsk`)
- Implement single `open_position` function that accepts `Strategy` parameter
- Implement single `add_liquidity` function that accepts `Strategy` parameter
- Use pattern matching on `Strategy` enum to apply strategy-specific weight formulas
- Extract common logic into helper functions:
  - `calculate_distribution_params`: handles weight calculation for all strategies
  - `calculate_active_weights`: computes active bin weights based on strategy

Remediation

✓ **Commit:** 05a78aa

About the Auditor

Asymptotic provides a white-glove, formal-verification-based auditing service for Sui smart contracts.

- Analyze customer codebases to create mathematical proofs of security
- Use AI to accelerate specification writing, proof construction, and adjusting specs as code changes
- Focus on real-world security properties that matter for production systems

Audit Team

Lead Auditor
Mykhailo Burakhin — Principal Auditor

- Conducted detailed security audits and gas optimizations for complex DeFi applications
- Developed and audited ERC-20, ERC-721, and ERC-1155 token contracts, including staking, vesting, auctions, and launchpads
- Engineered cross-chain bridges using ERC-4337 account abstraction, multi-signature wallets, and upgradeable proxy contracts
- Implemented protections against anti-sniping and front-running, and secure onboarding with MPC and zero-knowledge proofs
- Over 15 years of experience architecting secure, high-performance systems with a strong focus on cryptography, blockchain security, and advanced C++ engineering